Toronto Metropolitan University

Department of Electrical, Computer and Biomedical Engineering

Faculty of Engineering and Architectural Science

Winter 2026 Capstone Design Report: Pick-and-Place Three-Axis Robotic Arm

by

Rafael Capela-Rocha, Luca Valitutti, Katrina Kezele, and Jacob Mohamed

Computer/Electrical Engineering Capstone Design Project

Toronto Metropolitan University, Year 2026

Acknowledgements

We would like to express my deepest appreciation to our Faculty Lab Coordinator, Meranda Salem, for their continuous support and guidance throughout the duration of this project. Their role as mentors was invaluable, providing us with timely and constructive feedback that helped shape and refine our design at every stage. Meranda's willingness to answer questions, review our progress, and challenge our ideas contributed greatly to both our technical development and overall learning experience. Their encouragement and expertise created an environment where our team could confidently explore solutions and improve upon our work.

We would also like to extend my sincere gratitude to our course coordinator, Dr. Naimul Mefraz Khan, whose dedication to the course played a significant role in the successful completion of this capstone. Dr. Khan organized and delivered lectures that supported each step of the design process, offering practical strategies, insightful tips, and clear guidance to help us navigate complex engineering challenges. In addition, he organized seminars on proper lab safety to ensure that our team could conduct all design and testing activities responsibly and effectively. His commitment to student learning and his approachable teaching style made a meaningful difference in our ability to complete this project with confidence.

Additionally, we would like to acknowledge the efforts and collaboration of all team members, Rafael Capela-Rocha, Luca Valitutti, Katrina Kezele, and Jacob Mohamed. Each member contributed unique skills, ideas, and perspectives that were essential to the development and completion of this project. The teamwork, communication, and collective problem-solving within the group were fundamental to overcoming challenges and achieving our final design.

We are sincerely grateful to everyone who supported and contributed to the success of this capstone project.

Certification of Authorship

We certify that the work presented in this capstone final report for the Pick and Place Three-Axis Robotic Arm is the result of our own effort and original research. All information, data, and sources used in the development of this report have been properly cited and acknowledged. This report has not been submitted, in whole or in part, for credit in any other course or academic activity. We affirm that the work complies with the academic integrity policies of the Department of Electrical, Computer, and Biomedical Engineering and Toronto Metropolitan University.

Table of Contents

Acknowledgements 2
Certification of Authorship 3
Abstract 5
Introduction & Background 6
Objectives 7
Theory and Design 9
    Theory 9
    Design 10
        Overview 10
        Base and Arm Assembly 15
        Jamming Gripper 22
Alternative Designs 24
Materials/Components List 25
Measurement and Testing Procedures 26
Performance Measurement Results 27
    Power Source 27
    Object Pickup 27
Analysis of Performance 28
Conclusions 29
References 30
Appendices 31

Abstract

This project presents the design and implementation of a three-axis pick-and-place robotic arm with an integrated suction-based gripper. The objective was to develop a low-cost system capable of reliable object manipulation through both manual and automatic control modes. The system consists of a 3D-printed robotic arm driven by servo motors and controlled using two Arduino microcontrollers, enabling coordinated arm motion and suction control. The gripper utilizes a jamming mechanism composed of a latex balloon filled with coffee grounds and a filter dividing the suction and coffee grounds, providing adaptive gripping of various objects. System integration included communication between the two microcontrollers and the development of control algorithms for repeatable motion sequences. During testing, challenges such as instability, insufficient joint torque, and synchronization issues were identified and resolved through hardware redesign/upgrades, improved power delivery, and refined control logic.

The final system demonstrates consistent and repeatable pick-and-place performance with improved stability and control accuracy. Overall, the project successfully meets the intended design objectives and highlights the effectiveness of integrating mechanical design, embedded systems, and control strategies in robotic applications.

Keywords: Jamming gripper mechanism; Pick-and-place robotics; Arduino-based control systems; Embedded systems integration; Robotic arm automation.

Introduction & Background

Pick-and-place robotic systems are widely used in modern automation and manufacturing due to their ability to improve precision, repeatability, and efficiency in object handling tasks. These systems are particularly valuable in environments where repetitive or hazardous operations are required, reducing human error while increasing productivity. This project presents the completed design and implementation of a compact, low-cost three-axis robotic arm that demonstrates key principles of embedded control and robotic manipulation.

The system consists of a three-axis arm driven by servo motors, along with an additional servo for base rotation, enabling flexible positioning and object transfer. The mechanical structure was designed using CAD and fabricated through 3D printing, allowing for rapid prototyping, lightweight construction, and ease of customization. Control of the system is implemented using an Arduino Uno microcontroller, selected for its ease of use, sufficient I/O capabilities, and compatibility with standard control libraries.

A key feature of this design is the implementation of a suction-based soft robotic gripper inspired by the universal jamming gripper concept[1]. The gripper uses a latex balloon filled with granular material that conforms to objects when vacuum pressure is applied and releases them when pressure is restored. This mechanism is achieved using two low-voltage air pumps controlled through a secondary Arduino-based pump control module with an attached pump driver extension.

The final system integrates mechanical, electrical, and embedded components into a fully functional prototype capable of performing basic pick-and-place operations. The robotic arm was successfully fabricated, assembled, and tested, demonstrating effective coordination between servo actuation and pneumatic control. This report presents the completed design, implementation process, and performance evaluation of the system.

Objectives

The overall objective of this project was to design and implement a cost-effective, efficient three-axis robotic arm capable of performing automated pick-and-place tasks. The system was developed with a focus on positional accuracy, repeatability, and reliable operation under practical power constraints.

The key technical objectives of the completed project are outlined below:

- Mechanical Design and Fabrication
    - Design a three-axis robotic arm with an additional base rotation using CAD tools
    - Fabricate all structural components using 3D printing for rapid prototyping and lightweight construction
- Embedded Control System
    - Implement motion control using an Arduino Uno microcontroller
    - Program precise and repeatable servo positioning for all joints
- Dual Operation Modes
    - Implement a push-button interface to switch between manual and automatic modes
    - Automatic Mode:
        - Execute predefined pick-and-place sequences using hard-coded positions
        - Signal to the secondary Arduino to grab and release objects at predetermined states
    - Manual Mode:
        - Allow user control of each servo motor using potentiometers
        - Enable user-controlled gripping and releasing via a push button
- Gripper and Pneumatic System
    - Develop a suction-based soft robotic gripper using a latex balloon filled with granular material
    - Enable the gripper to securely grasp, hold, and transport objects without loss of grip

    - Integrate air pumps controlled via a secondary Arduino with a pump driver extension
- System Operation and Performance
    - Ensure the robotic arm can repeatedly perform pick-and-place operations with consistent results
    - Achieve reliable coordination between motion control and gripper actuation
    - Maintain stable operation during continuous task execution
- Power and Electrical Design
    - Design a power supply capable of delivering sufficient current to maintain torque across all servo motors
    - Ensure stable operation of both servo actuation and pneumatic components without performance degradation

Theory and Design

Theory

Arduino is the platform of choice for the implementation of the robotic arm due to its wide assortment of parts and extensions available for it. The code used for the project is written in C++ using the Arduino IDE.

The following equations were used to model the arm's kinematics and determine joint angles corresponding to desired positions:

$$x = [L_1 cos\theta_2 + L_2 cos(\theta_2 + \theta_3)] cos\theta_1 \text{ (x-direction)} \quad (1)$$

$$y = [L_1 cos\theta_2 + L_2 cos(\theta_2 + \theta_3)] sin\theta_1 \text{ (y-direction)} \quad (2)$$

$$z = L_1 sin\theta_2 + L_2 sin(\theta_2 + \theta_3) \text{ (z-direction)} \quad (3)$$

No kinematic equation was provided for the base joint as it only controlled the rotation of the entire arm assembly. This joint would simply be rotated between 0° and 180°.

The inverse kinematics equations are:

$$\theta_1 = arctan2(y, x) \text{ (shoulder rotation)} \quad (4)$$

$$r = \sqrt{x^2 + y^2} \text{ (distance from base to target in x-y plane)} \quad (5)$$

$$D = \frac{r^2 + z^2 - L_1^2 - L_2^2}{2L_1 L_2} \quad (6)$$

$$\theta_3 = atan2(\sqrt{1 - D^2}, D) \text{ (wrist angle)} \quad (7)$$

$$\theta_2 = atan2(z, r) - atan2(L_2 sin\theta_3, L_1 + L_3 cos\theta_3) \text{ (elbow angle)} \quad (8)$$

Equations (1), (2), and (3) determine the gripper's location using θ_1, θ_2, and θ_3 and the lengths of the arm segments L_1, L_2, and L_3, and equations (4) through (8) calculate the arm's end position given the desired x, y, and z coordinates of the position.

Design

Overview

The robotic arm has two modes: automatic and manual mode. By default, when the arm is first turned on, it initializes to the default position before performing automatic operation. A button may be pressed to switch between modes. An overview of the full robotic arm operation is shown in **Fig. 1.**

In automatic mode, the arm positions the gripper over the object, then it goes on top of the object and initiates the pump's suction sequence. It then moves the arm to the destination position and releases the object. This sequence of events is then repeated until the user wishes to switch modes. The two modes are also indicated visually using a single LED, which is on for manual mode and off for automatic mode.

In manual mode, the pump reads several inputs: three potentiometers which control the joints of the robotic arm and one potentiometer controlling the base, as well as a button to trigger the pump to inhale or exhale. With full control over the robotic arm, it may perform any sequence of actions.

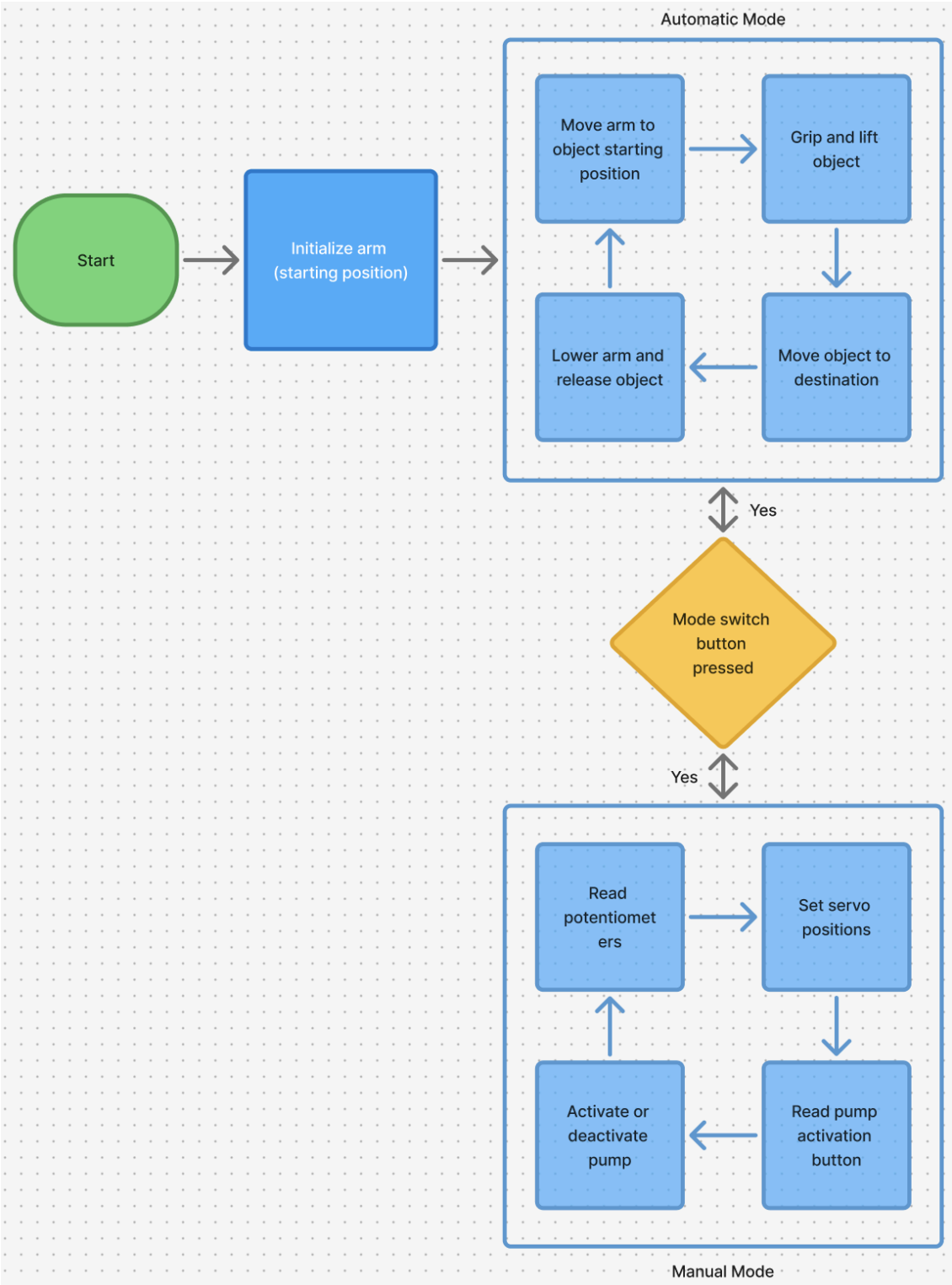


Figure 1. Automated robotic arm operation flow chart

At the heart of the robotic arm are two Arduino Uno R3s. All components used in the design are connected to the two Arduinos, hence their center position in the block diagram shown in **Fig. 2.** below.

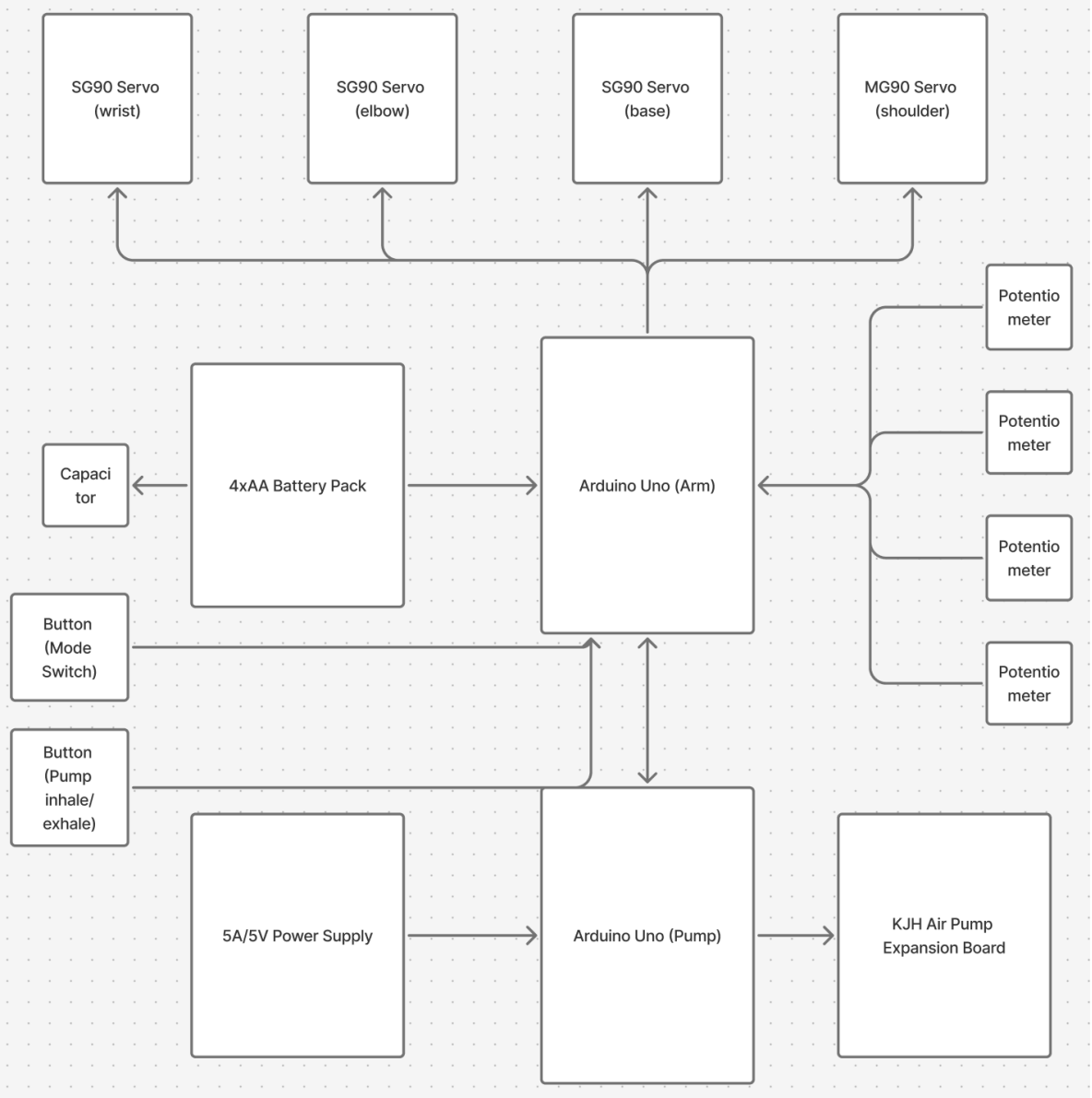


Figure 2. Schematic of robotic arm and its components

The main Arduino board, labelled Arduino Uno (Arm) is the heart of the entire system. It controls the servos, which are powered by a 4xAA battery pack, and contains a capacitor between its power supply and ground to reduce jitter due to excessive current draw. The 4xAA battery pack was chosen due to its ability to hold 4 rechargeable batteries each having 1.2-1.4

volts for a total series voltage of 4.8-5.6V. The Arduino Uno (Arm) controls the servos and receives signals from the potentiometers and the two buttons. It also sends signals to the other Arduino labelled Arduino Uno (Pump).

That Arduino, used for control of the pump, is powered by a 5A/5V power supply, and controls the KJH Air Pump Expansion Board, which is responsible for inhaling and exhaling the balloon. A full schematic with all of the components is shown in **Fig. 3.**

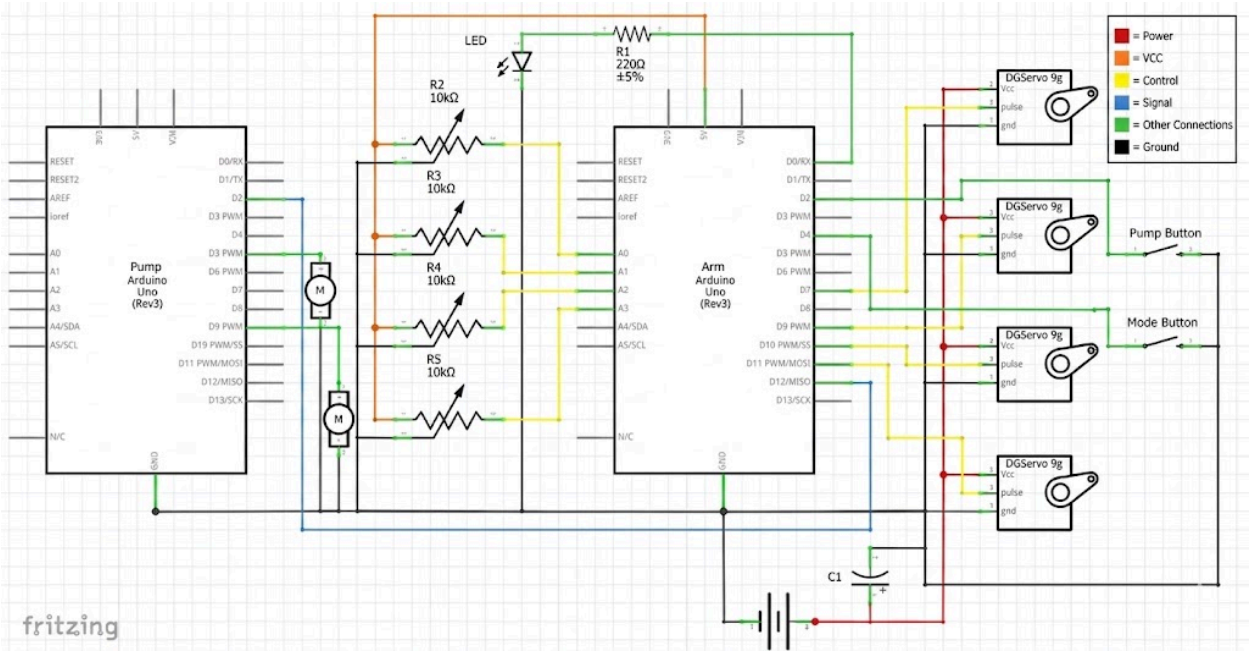


Figure 3. Fritzing schematic of full robotic arm assembly

The design of the robotic arm required a reliable and organized electronic control system to ensure proper coordination between the components and control logic. Using Fritzing to design this schematic, it gave a clear visualization of how each electrical component interfaces with the microcontroller. While earlier stages of the project focused on the mechanical structure of the arm, this phase emphasized the electrical architecture needed to support the movement of the arm and the manipulation of our grabber.

The completed electronic design consists of four servo motors, two air pumps, two Arduinos, two push buttons, four potentiometers, one capacitor, one resistor, and one LED. One of the Arduinos will act as the master control system and will be in charge of controlling the

overall processes of the arm. The second Arduino will act as the slave and will have a dual pump extension board and will be responsible for controlling the pumps. When the master Arduino wishes to inhale or exhale the pumps, it simply sends a high signal to one of its digital pins, which is then read by the pump arduino. The pump Arduino will then control the pumps according to that signal.

The four servo motors are responsible for controlling the rotational movements of the joints of the arm and the base, enabling precise positioning. The two air pumps are used to inflate and deflate a latex balloon filled with coffee grounds, which will function as the gripping mechanism for picking up and releasing objects. The two Arduinos communicate with each other using a single wire across two digital pins(One from the Arduino with the pump extension and one from the Arduino for control).

The robotic arm is programmed with two primary functions: *runAutomaticMode()* and *runManualMode()*. Upon startup, the system executes the setup routine to initialize all required components. Once initialization is complete, the program enters *runAutomaticMode()*, where the arm continuously cycles through three operational phases.

In the first phase, the arm moves to a predetermined position using servo values established through prior testing, then lowers the gripper to engage the target object. In the second phase, the arm's microcontroller sends a signal to the pump controller, instructing it to initiate suction (inhale). While suction is maintained, the arm proceeds to the third phase, moving laterally to the designated drop-off location. At this point, a signal is sent to the pump controller to release suction (exhale), allowing the object to be placed.

The system includes a mode-switching mechanism controlled by a button. When pressed, the program exits the automatic loop and calls the *moveToDefault()* function, which returns all servos to their home positions. The *checkDefaultComplete()* function verifies that this reset process has finished before transitioning the system into *runManualMode()*.

In manual mode, an LED indicator is activated to signal the change in operation. The system continuously reads input from potentiometers, allowing the user to directly control servo

positions in real time. Additionally, a secondary button enables manual control of the pump: one press activates suction, and a subsequent press deactivates it.

The user can press the mode-switch button again to return to automatic mode. Upon doing so, the arm is first returned to its default position via *moveToDefault()*, after which it immediately resumes the automatic sequence, beginning with the first phase and continuing through the full object manipulation cycle.

Base and Arm Assembly

The mechanical design of the three-axis robotic arm was developed around the use of three SG90 micro servos, one MG90 servo and 3D-printed structural components. These servos were selected due to their low cost, compact size, and suitability for lightweight robotic applications. The arm consists of three rotational axes, each driven by an SG90 servo, with the exception of the shoulder joint, enabling controlled movement in multiple planes. The MG90 servo was chosen to control the shoulder joint due to its increased torque capacity and its metal gears, as opposed to the plastic gears that the SG90 servos have. This decision was made due to the initial testing of the arm, which resulted in failure of the shoulder joint to allow the arm to move smoothly. All structural components were designed using CAD software with a focus on modularity, repeatability, and ease of manufacturing through 3D printing. A standardized servo mounting strategy was adopted throughout the design. As shown in **Fig. 4**, the servo mount dimensions were defined based on the SG90 datasheet and then reused across all other servo mounts presented in subsequent figures. Maintaining identical dimensions for each mount ensures consistency across the robotic arm and simplifies both assembly and future modifications. This approach also allows any servo mount to be replaced interchangeably without requiring redesign or reprinting of adjacent components.

Particular attention was given to achieving a snug fit between the servos and the 3D-printed mounts. The design process involved iterative refinement of the servo cavity dimensions to minimize mechanical play while still allowing the servos to be inserted without excessive force. By closely matching the servo geometry, the resulting tolerance between the servo housing and the printed part is minimal, reducing vibration and backlash during operation. This tight fit improves positional accuracy and overall mechanical stability of the robotic arm. To optimize the manufacturability of the components, certain parts were intentionally separated into

multiple sections. **Fig. 8.** and **9.** illustrate components that were split to reduce overhangs during the 3D printing process. By separating these parts, the need for extensive support material was minimized, resulting in cleaner prints, reduced material usage, and shorter print times. This design decision also improves surface quality and dimensional accuracy, which is especially important for components that interface directly with the servo mounts.**Figs 5 - 9** showcase all of the designs for each segment and part of the arm along with their dimensions. The entire arm's design could be seen in **Fig. 10.**

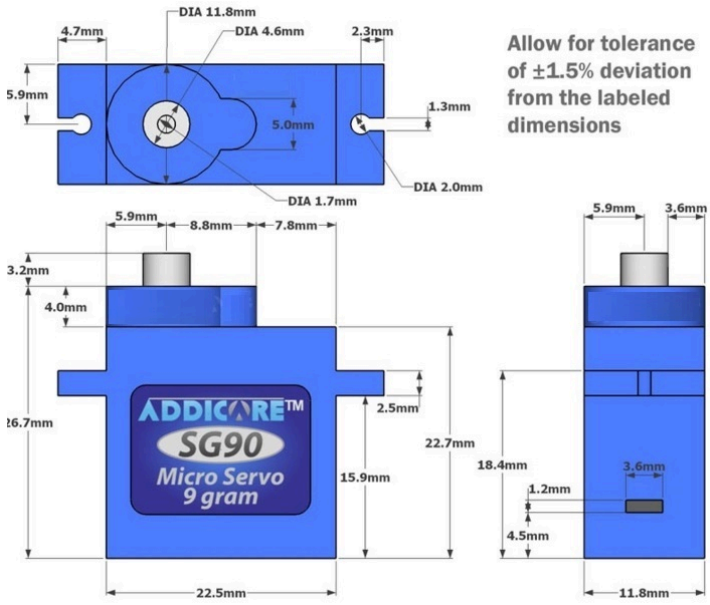


Figure 4: SG90 dimension sheet

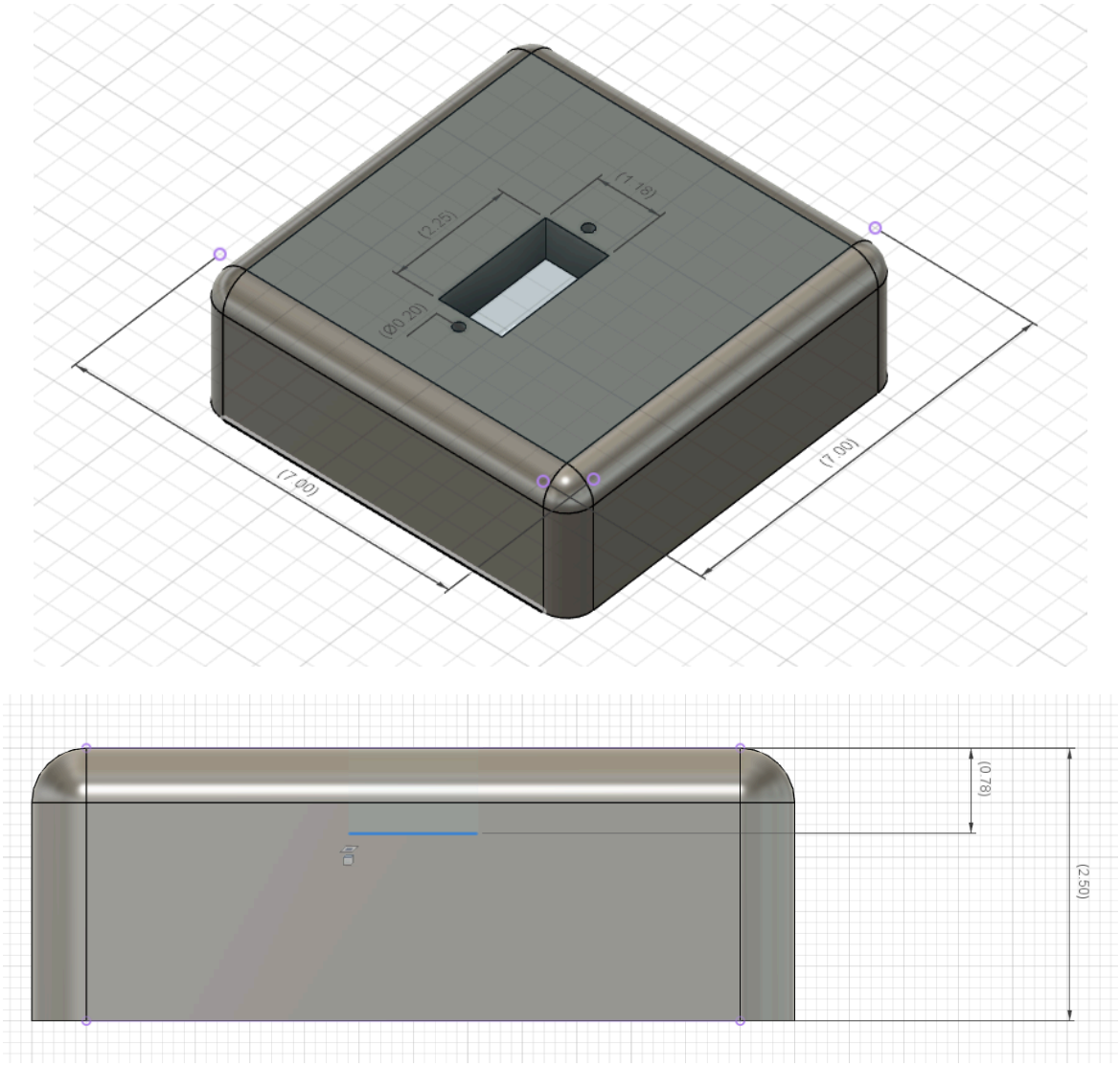


Figure 5. 3D models of the robotic arm base

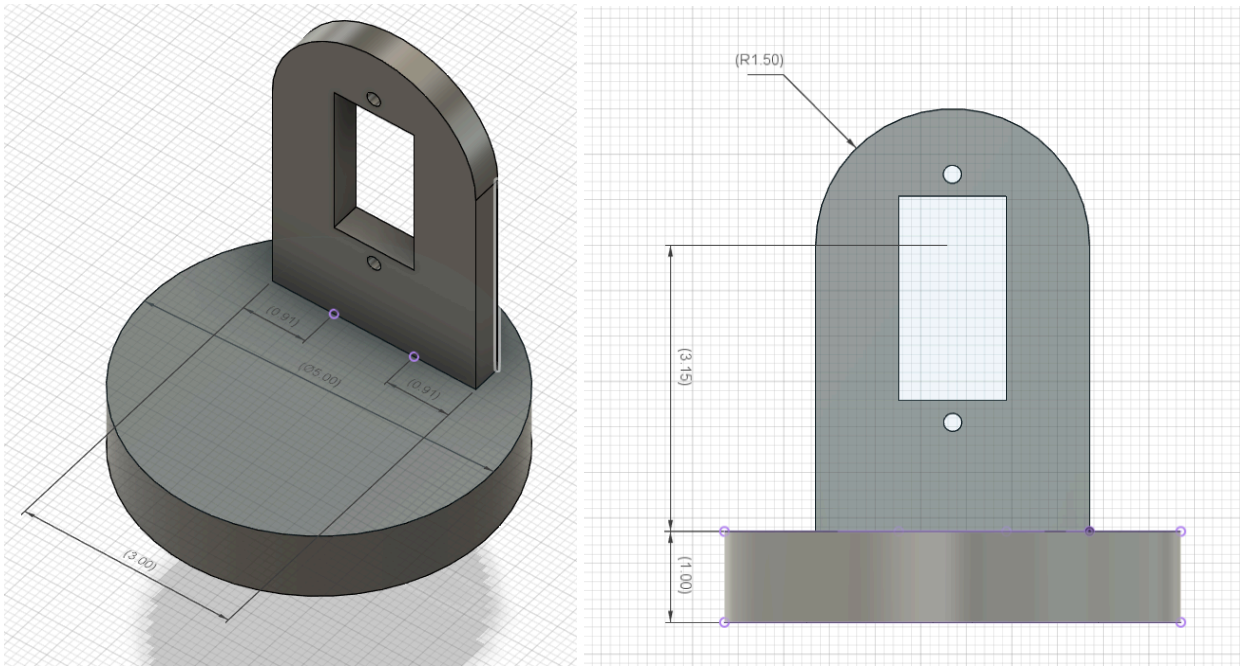


Figure 6. 3D model of the robotic arm rotational base

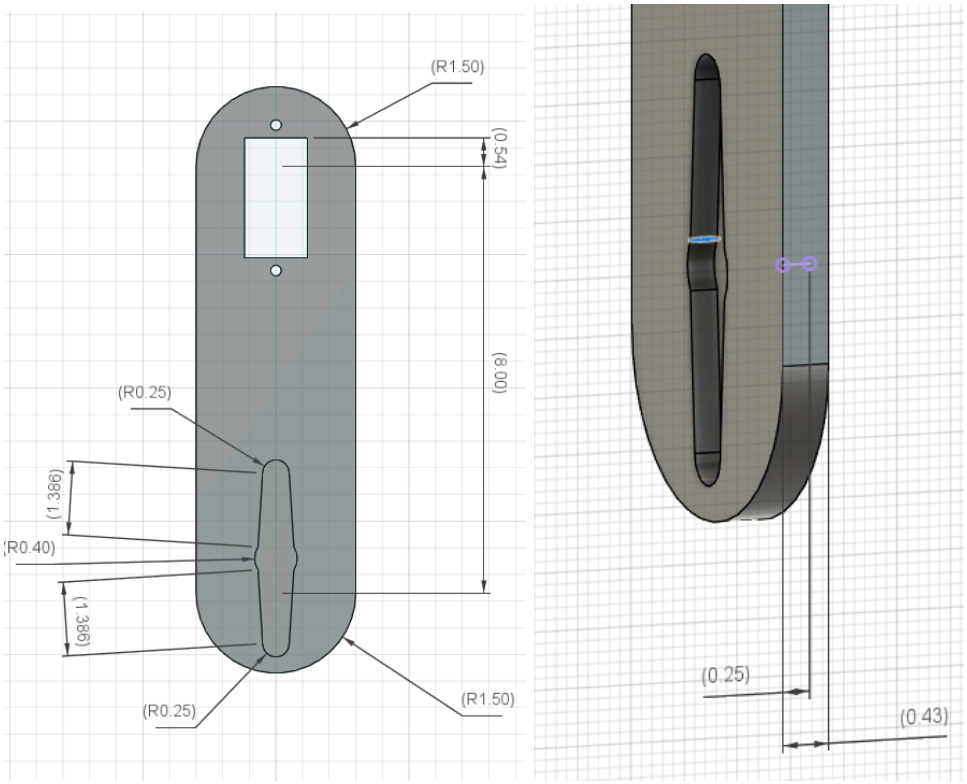


Figure 7. 3D model of the robotic arm segments

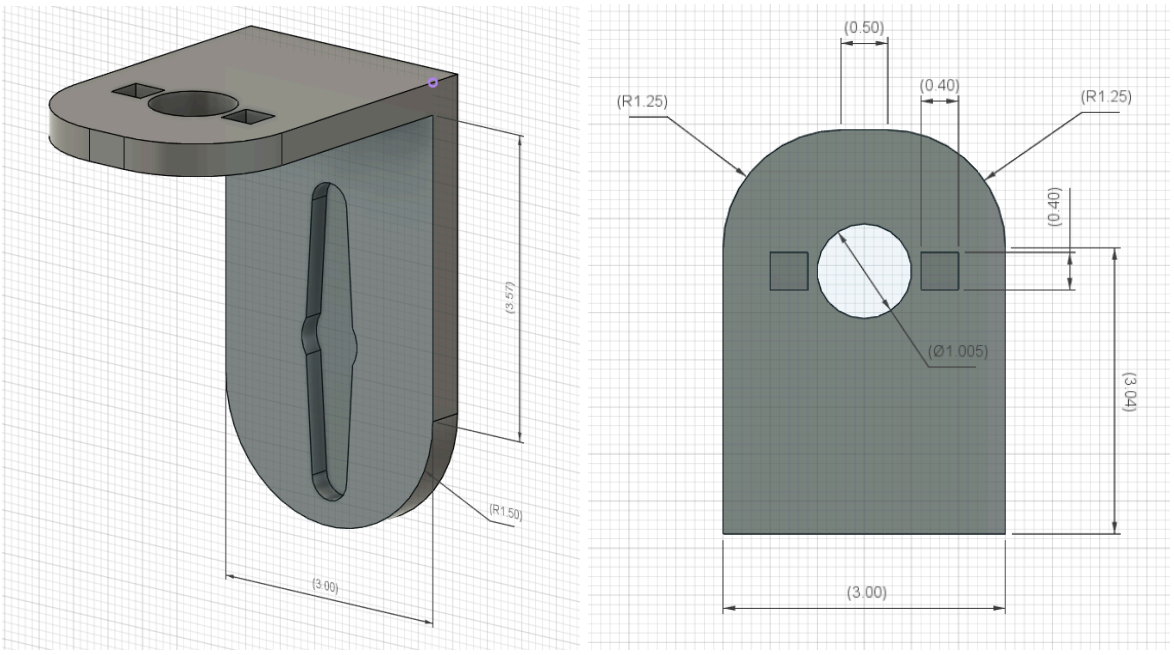


Figure 8. 3D model of the uppermost segment of the robotic arm

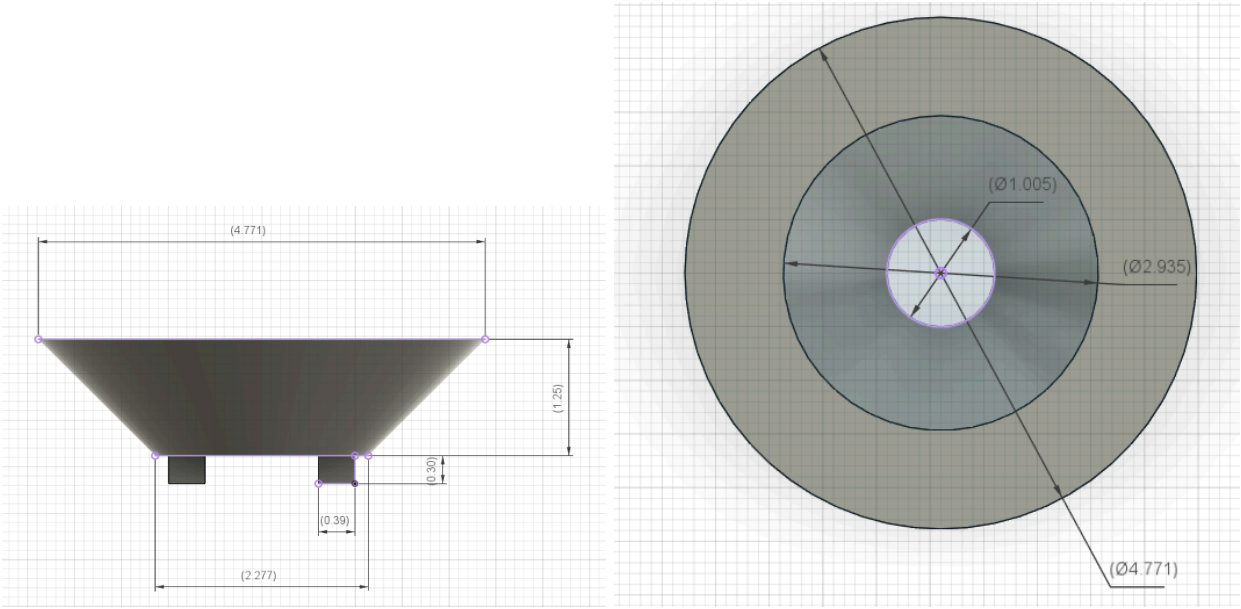


Figure 9. 3D model of the grabber segment

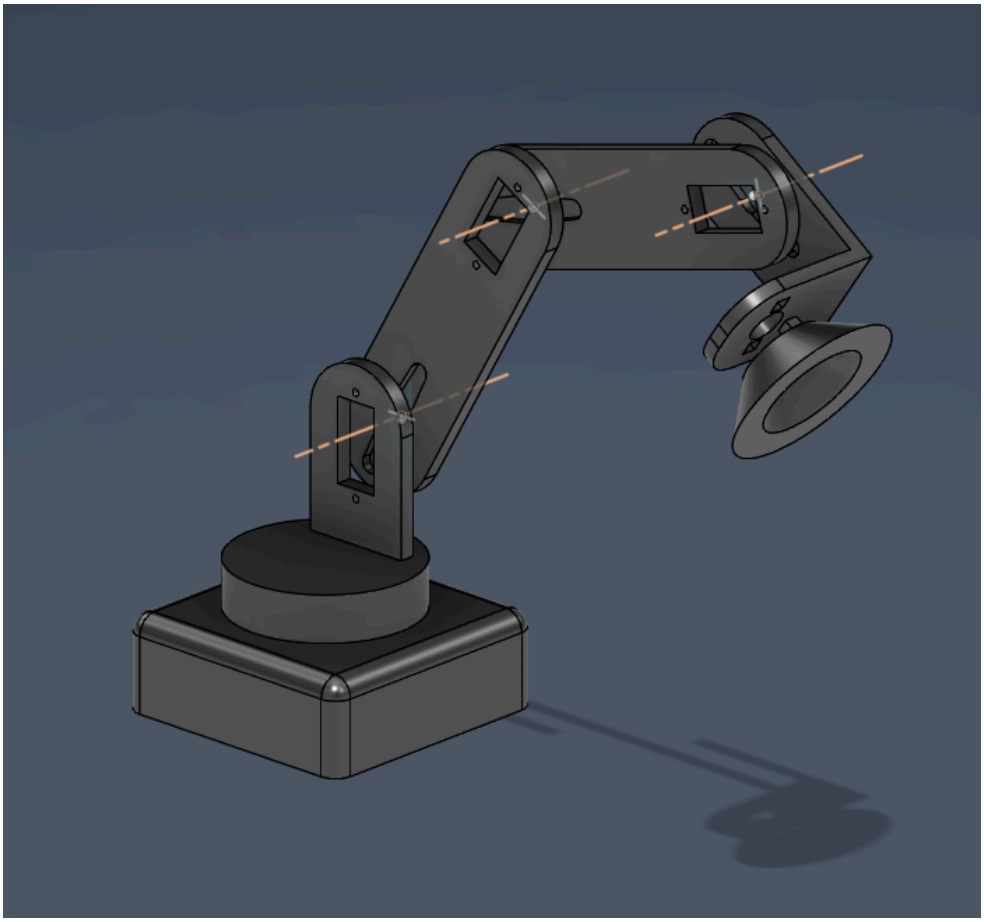


Figure 10: 3D model of the full arm design

For movement of the arm segments, servo motors are used due to their closed-loop feedback system and quick response time. Each servo contains an internal feedback controller, simplifying position control without requiring external encoder feedback. One servo motor is located at each joint of the arm, and one at the base, for a total of 4 servo motors. The servos used in the rotational base, elbow, and wrist are Addicare SG90s as seen in **Fig 11**.



Figure 11. Addicare SG90 Micro Servo

For the servo in the shoulder of the Arm, a Tower Pro MG90s servo(seen in **Fig. 12**) was used as it would be supporting the weight of the entire arm assembly. The MG90 was chosen for this joint over an SG90s due to a previous failure using the SG90s, where there was too much torque on the servo, making it unable to raise the entire arm.



Figure 12. Tower Pro MG90s Micro Servo

Each of the servo motors draws about 3-7.2A, depending on load, and will be powered through the 4xAA battery pack through the main Arduino Uno. A few more specifications of both servos are tabulated below in **Table 1** for reference.

Criteria	SG90s	MG90s
Voltage Range (V)	4.8-6	4.8-6
Weight (g)	11	14
RPM Range	67-84	100-125
Power Range (W)	1.1-1.5	2.2-3.4

Table 1. Specifications of SG90s vs MG90s Micro Servos

Jamming Gripper

The jamming gripper assembly consists of three main components – the balloon filled with coffee grounds, the 3D printed mount attaching it to the rest of the arm, and the pump system. The interface between the former two contains a filter to prevent grounds from being sucked into the pump, and the latter is located near the base of the Arduino board. Silicone tubing will be run through the arm from the expansion board up to the gripper mount to provide suction to the mechanism.

The pump assembly controlling the suction of the gripper is located on an expansion board, which consists of two air pumps, a three-way solenoid valve, motherboard leads, and a DC power input. **Fig. 13,** shown below, shows the pump expansion board used to control the pumps.:

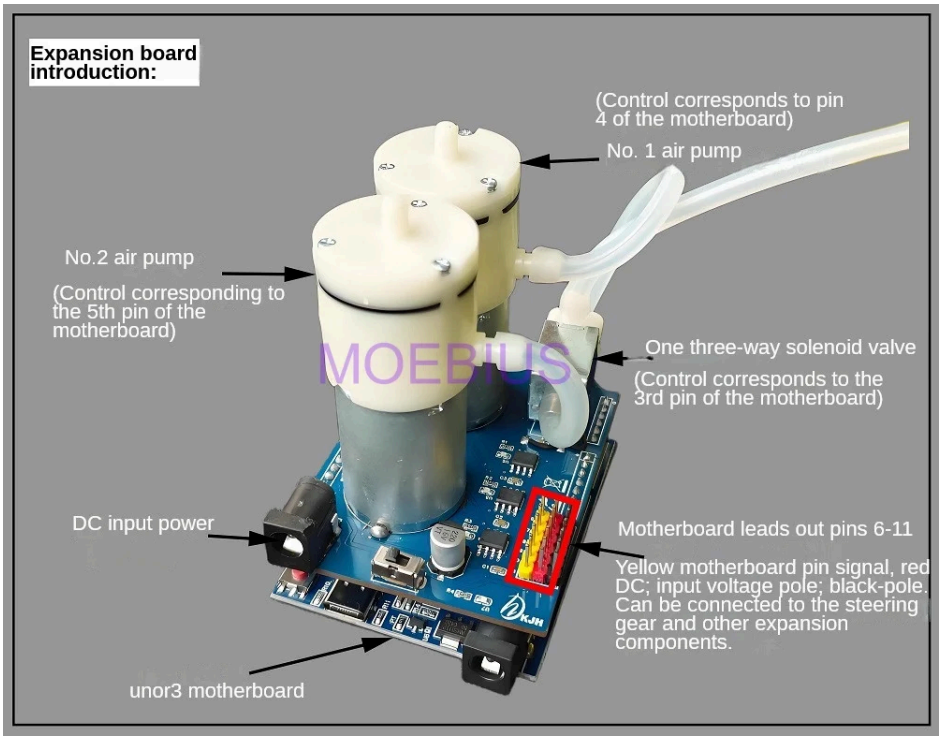


Figure 13. Arduino pump expansion board diagram

A 5V power supply is used to power the device as the voltage is rated at 4.5-8.4V, and the power will be fed by a universal power supply providing power to the entire arm assembly. Several other specifications for the pump are tabulated below for reference in **Table 2**.

Criteria	Specification
Rated DC Voltage (V)	4.5-8.4
No-load Current (mA)	>750
Maximum Vacuum Pressure (kPa)	-58
Air Flow (L/min)	3.0
Pressure Range (mmHg)	350

Table 2. Air pump specifications

Through thorough testing of the air pump with the robotic arm, several bases were covered including pump overheating, balloon leakage/rupture, and coffee grounds entering the

pump. It was found that under regular load, the pump did not overheat. It was tested for ~50 consecutive cycles, and only showed minor signs of warmth, but not enough to warrant any significant concern. During testing, the balloon also did not show any signs of rupturing. This was due to a thicker balloon being used. Although it did reduce the dexterity of the gripper, it ensured a reliable material for the gripper. Another concern was coffee grounds entering and compromising the pump system. To combat this, a simple glass-cleaning microfiber cloth was wrapped around the end of the tubing that went inside the balloon, and a ziptie was used to keep the three parts together reliably.

Alternative Designs

An alternative design considered for the actuation system was the use of all MG90s metal gear servos instead of SG90 plastic gear servos. MG90s servos provide higher torque and improved durability due to their metal gear construction, enabling them to support heavier loads and increased repetitive motion. This alternative was explored after the SG90 servo at the base of the arm failed due to carrying the load of the entire arm. It was replaced with a more reliable MG90s servo and instantly showed improved performance and response time. Implementing MG90s servos at each joint of the arm would increase its overall reliability and performance. Despite its slightly higher price of about $1-2 per servo, it would greatly increase the strength, responsiveness, and durability of each of the joints.

Another alternative design explored involved the coffee grounds in the balloon used in the gripper system. Deciding between instant coffee and ground coffee was difficult, since both types presented their own advantages and disadvantages. Instant coffee, due to its dryness and sparse consistency, was a good candidate for the gripper balloon as it allowed for greater flexibility and reduced the likelihood of particles clumping together after repeated pressure cycles. However, due to the repetitive nature of pressure changes in the balloon, the instant coffee particles would gradually break down into finer particles over time. Regular ground coffee was also considered as an alternative because its finer consistency could allow the gripper to conform more closely to objects, improving grip. However, its fine particle size caused it to pack too densely under suction, which reduced flexibility and made it more likely to form clumps during repeated use. In the end, a mix of hand-ground instant coffee particles and unmodified instant coffee particles was used. This combination maintained the flexibility of larger particles

while incorporating finer particles to improve grip strength, resulting in a better balance between dexterity and rigidity when interacting with objects.

Materials/Components List

The selected components were chosen based on a balance between cost, efficiency, reliability, and compatibility with the overall system design. Low-cost and widely available components like the Arduino Uno and SG90s servo motors were selected to ensure ease of implementation and replacement. However, the servo attached to the shoulder joint of the robotic arm was replaced with MG90s, which use metal gears, as it would be supporting the weight of the entire robotic arm assembly. The chosen servo motors provide sufficient torque and positional control for lightweight objects, designating them suitable for the intended application.

The pump and valve system was selected to support the suction-based jamming gripper, enabling the robotic arm to handle objects of varying shapes and sizes. Electrical components such as capacitors, the power supply, and the batteries were chosen to ensure stable operation while protecting the system from voltage fluctuations and excessive current draw.

While higher-performance components were considered, the final selection prioritized simplicity and cost-effectiveness. These design choices allowed for a functional and reliable design while remaining within project constraints.

The full list of materials and components used in the final design is tabulated below in **Table 3**.

Item	Quantity	Cost ($)
Air Pump/Tubing/Valve	1	52.49
Arduino Uno	2	11.3
Balloons	1	1.7
Batteries, AA	4	27.92
Battery Holder	1	3.64
Buttons	2	1.63

Capacitor (1mF)	2	2.26
Coffee Grounds	1	1.41
Microfiber Cloth	1	1.62
NiMh Battery Charger	1	13.4
Potentiometer	4	2.28
Power Supply	1	20
Servo, MG90	1	1.62
Servo, SG90	3	4.31
Totals	23	145.58

Table 3: Price and quantity of all materials and components used in the project

As shown in the table above, the total cost of the project equated to $145.58, which is less than half of our expected budget of $300. Because of our conservative parts selection from cheap and even free sources like AliExpress, Amazon, old lab kits, and the Engineering Building, we were able to minimize costs while also producing a fully functional and successful model of our robotic arm.

Measurement and Testing Procedures

(i.e. simulation procedures w2021)

It is essential that a project be properly designed. A designer must satisfy the examiner, the FLC, that the program or circuit will perform its tasks to specification under all or at least the usual, variations in (simulated) operating environment.

Another guide used to assess whether the design is competent is to consider the mass production of this prototype. Could one anticipate a reasonable yield and customer satisfaction?

The procedure used to measure performance is to be described in sufficient detail that the reader can repeat it. The measured results must be documented in conjunction with appropriate schematics or flow charts.

The results should be analyzed to ensure that they fit the anticipated performance and if not an explanation is called for.

Performance Measurement Results

The performance of the developed 3-axis pick-and-place robotic arm was evaluated through a series of experimental tests with varying operating conditions and objects. Key performance metrics, including positioning accuracy, repeatability, and autonomy, were measured to assess the system's effectiveness. The experiments were conducted using different objects and voltages to observe how these factors influence the overall behaviour of the robotic arm. The results obtained from these tests are presented in this section below.

Power Source

The robotic pick-and-place arm was powered using AA battery configurations to evaluate system performance under different supply voltages. Initially, the servos were powered using three 1.5 V AA batteries (4.5 V total). Under this configuration, the system exhibited insufficient torque during high-load conditions, resulting in incomplete or unstable movements.

To improve performance, the power supply was modified to use four 1.2 V batteries (4.8V total). This configuration enabled the servos to operate more consistently, particularly during tasks requiring higher torque, such as lifting heavier objects.

Object Pickup

The robotic arm was tested for its ability to pick up objects of varying shapes and sizes using a universal jamming gripper. The system successfully handled a range of objects; however, performance limitations were observed in the shoulder joint.

During lifting tasks, the shoulder servo exhibited noticeable jitter, characterized by small oscillatory movements under load. After replacing the SG90s servo with an MG90s metal gear servo and adding a 1000 μF capacitor to the power supply, the system demonstrated improved stability, with reduced jitter during object lifting operations.

Analysis of Performance

The experimental results highlight the impact of power supply and servo selection on the performance of the robotic arm. The initial use of three 1.5 V batteries provided insufficient voltage and current for the servos, particularly under high torque conditions. This resulted in unstable or incomplete movements, indicating that the power supply was unable to meet peak current demands.

The improved performance observed with the four 1.2 V battery configuration suggest that even a small increase in voltage can significantly enhance servo operation. This improvement is likely due to the increased ability of the system to supply sufficient current during high-load conditions, reducing voltage drops during operation.

The jitter observed in the shoulder servo during object pickup can be attributed to both mechanical and electrical limitations. Mechanically, plastic gear servos are more prone to deformation and backlash under load which can lead to instability. This was even proven during experiments where the servo produced audible clicks, likely wearing down the gear. Electrically, rapid current fluctuations from the servo can introduce noise and voltage dips in the system.

The replacement of the servo with a metal gear MG90s improved mechanical stability, while the addition of a 1000 μF capacitor helped regulate voltage by supplying instantaneous current during peak demand. This reduced jitter and improved the overall smoothness of operation.

Overall, the results indicate that both adequate power supply design and appropriate servo selection are critical for achieving stable and reliable performance in a three-axis robotic arm system.

Conclusions

This project successfully achieved the design and implementation of a three-axis pick-and-place robotic arm integrated with a suction-based jamming gripper. The system combines a 3D-printed mechanical structure, servo motor actuation, and a dual-Arduino control architecture to enable coordinated arm movement and object manipulation. Both manual and automatic control modes were developed, allowing for user interaction as well as repeatable motion sequences. Through iterative design, testing, and refinement, the final system demonstrated reliable and consistent pick-and-place functionality.

The initial project objectives focused on developing a low-cost, stable, and functional robotic system capable of accurately manipulating small objects. These objectives were largely met; however, some discrepancies arose during development. For example, the original design underestimated the torque requirements at the shoulder joint and the need for a more stable base. These issues required hardware upgrades, including stronger servo motors and improved mounting solutions, as well as modifications to the mechanical design.

Several challenges were encountered throughout the project, including synchronization between the robotic arm and suction system, reliable communication between the two Arduino boards, and optimization of the jamming gripper material composition. While most of these issues were resolved through iterative testing and code refinement, minor limitations remain in achieving perfectly smooth motion and fully optimized grip consistency under all conditions.

Future work in this area can build upon the insights gained from this project by focusing on improving system precision, reliability, and integration. The implementation of closed-loop control using sensors could enhance positioning accuracy and reduce dependence on pre-programmed motion sequences. Additionally, simplifying system architecture by reducing reliance on multiple microcontrollers or improving communication protocols could increase efficiency and robustness. Further refinement of adaptive gripping mechanisms, such as optimizing material composition and pressure control, would improve versatility across a wider range of objects. These considerations provide a foundation for future designs seeking to develop more advanced and scalable robotic automation systems.

References

[1] E. Brown, C. Rodenburg, J. Amend, et al., “Universal robotic gripper based on the jamming of granular material,” Proc. Natl. Acad. Sci., vol. 107, no. 40, pp. 18809–18814, Oct. 2010.

Appendices

robotic_arm_code

```
#include <Servo.h>

#define NUM_SERVOS 4

// ============================================================
// PIN ASSIGNMENTS
// ============================================================

Servo servos[NUM_SERVOS];

int servoPins[NUM_SERVOS] = { 6, 11, 10, 9 };
int potPins[NUM_SERVOS]   = {A0, A1, A2, A3 };

const int toggleButton = 2;
const int ledPin       = 4;
const int pumpButton = 3;

const int signalPin = 12;

// ============================================================
// SERVO ASSIGNMENTS
//
//   Index 0 = Base      (rotates whole arm)
//   Index 1 = Wrist
//   Index 2 = Elbow
//   Index 3 = Shoulder
// ============================================================

int servoMin[NUM_SERVOS] = {  0,   0,   0,   0 };
int servoMax[NUM_SERVOS] = {180, 180, 180, 180 };

// ============================================================
// POSITIONS
// ============================================================

int defaultPositions[NUM_SERVOS] = {90, 0, 0, 0};

#define MAX_PHASES 8
int autoPhase1[NUM_SERVOS] = { 140, 55, 32, 74}; //pickup
int autoPhase2[NUM_SERVOS] = { 141, 55, 32, 16}; //lift up
int autoPhase3[NUM_SERVOS] = { 36, 55, 32, 16};  //rotate and put object down

int* hardcodedPhases[] = {autoPhase1, autoPhase2, autoPhase3};
const int NUM_HARDCODED_PHASES = 3;

int recordedPhases[MAX_PHASES][NUM_SERVOS];
int recordedPhaseCount = 0;

// ============================================================
// PAUSE DURATIONS
// ============================================================

unsigned long phasePauseMS[] = {3000, 0, 3000};
//                               ^        ^
//                          phase 1   phase 3

// ============================================================
// CALIBRATION MODE
// ============================================================

const bool CALIBRATION_MODE = false;
```

```
// ============================================================
// RUNTIME STATE
// ============================================================

int currentPositions[NUM_SERVOS] = {0, 0, 0, 0};
int targetPositions[NUM_SERVOS]  = {0, 0, 0, 0};

int potBaseline[NUM_SERVOS]   = {0, 0, 0, 0};
int servoBaseline[NUM_SERVOS] = {0, 0, 0, 0};

bool manualMode      = false;
bool lastButtonState = HIGH;

unsigned long pressStartTime      = 0;
const unsigned long LONG_PRESS_MS = 1000;

bool pumpState = false;
bool lastPumpButtonState = HIGH;

const int stepDelay = 20;
unsigned long lastStepTime = 0;

// States: MOVING, PAUSING
enum AutoState { MOVING, PAUSING };
AutoState autoState  = MOVING;
bool autoRunning     = false;
int  activeServo     = 0;
int  activePhase     = 0;
unsigned long pauseStartTime = 0;

bool movingToDefault = false;

// ============================================================
// HELPERS
// ============================================================

int clamp(int value, int minVal, int maxVal) {
  return max(minVal, min(maxVal, value));
}

int* getPhase(int index) {
  return recordedPhaseCount > 0 ? recordedPhases[index] : hardcodedPhases[index];
}

int numPhases() {
  return recordedPhaseCount > 0 ? recordedPhaseCount : NUM_HARDCODED_PHASES;
}

void loadPhase(int phase) {
  int* p = getPhase(phase);
  for (int i = 0; i < NUM_SERVOS; i++) {
    targetPositions[i] = clamp(p[i], servoMin[i], servoMax[i]);
  }
  activeServo = 0;
  Serial.print("Auto: loading phase ");
  Serial.println(phase);
}

// ============================================================
// SETUP
// ============================================================

void setup() {

  Serial.begin(9600);
  pinMode(toggleButton, INPUT_PULLUP);
  pinMode(ledPin, OUTPUT);
  pinMode(signalPin, OUTPUT);
```

```
  pinMode(pumpButton, INPUT_PULLUP);

  digitalWrite(signalPin, LOW); // start with pump off

  delay(500);

  if (CALIBRATION_MODE) {

    Serial.println("CALIBRATION MODE: all servos at 90.");

    for (int i = 1; i < NUM_SERVOS; i++) {
      servos[i].attach(servoPins[i]);
      servos[i].write(0);
    }
    servos[1].write(90);
    return;
  }

  for (int i = 0; i < NUM_SERVOS; i++) {
    currentPositions[i] = clamp(defaultPositions[i], servoMin[i], servoMax[i]);
    targetPositions[i]  = currentPositions[i];
    servos[i].attach(servoPins[i]);
    servos[i].write(currentPositions[i]);
  }

  lastStepTime = millis();
  startAutomatic();
}

// ============================================================
// MAIN LOOP
// ============================================================

void loop() {

  if (CALIBRATION_MODE) return;

  handleButton();
  handlePumpButton();
  updateServos();

  if (movingToDefault) {
    checkDefaultComplete();
    return;
  }

  if (manualMode) {
    digitalWrite(ledPin, HIGH);
    runManualMode();
  } else {
    digitalWrite(ledPin, LOW);
    runAutomaticMode();
  }
}

// ============================================================
// POSITION RECORDING
// ============================================================

void recordCurrentPosition() {

  if (!manualMode) return;

  if (recordedPhaseCount >= MAX_PHASES) {
    Serial.println("Record: phase slots full.");
    return;
  }

  for (int i = 0; i < NUM_SERVOS; i++) {
```

```
    recordedPhases[recordedPhaseCount][i] = currentPositions[i];
  }

  Serial.print("Recorded phase ");
  Serial.print(recordedPhaseCount);
  Serial.print(": ");
  for (int i = 0; i < NUM_SERVOS; i++) {
    Serial.print(currentPositions[i]);
    if (i < NUM_SERVOS - 1) Serial.print(", ");
  }
  Serial.println();

  recordedPhaseCount++;
}

// ============================================================
// DEFAULT RETURN
// ============================================================

void moveToDefault() {

  movingToDefault = true;

  // Start with SHOULDER FIRST (index 3)
  activeServo = 3;

  // Set all targets
  for (int i = 0; i < NUM_SERVOS; i++) {
    targetPositions[i] = clamp(defaultPositions[i], servoMin[i], servoMax[i]);
  }

  Serial.println("Moving to default");
}

void checkDefaultComplete() {

  if (currentPositions[activeServo] != targetPositions[activeServo]) return;

  // shoulder > base > elbow > wrist
  if (activeServo == 3) activeServo = 0;   // after shoulder > base
  else if (activeServo == 0) activeServo = 1;
  else if (activeServo == 1) activeServo = 2;
  else if (activeServo == 2) {

    // DONE
    movingToDefault = false;
    Serial.println("Default reached.");

    if (!manualMode) {
      startAutomatic();
    } else {
      snapshotPotsForManual();
    }

    return;
  }
}

// ============================================================
//C MODE:    STATE MACHINE
//
// MOVING:   steps activeServo toward target one degree at a time.
//           When activeServo finishes, advances to next servo.
//           When all servos finish a phase, checks pause duration.
//           If pause > 0, switches to PAUSING state.
//           If pause = 0, loads next phase immediately.
//           After last phase, loops back to phase 0.
//
// PAUSING:  waits for phasePauseMS[completedPhase] to elapse,
```

```
//          then loads the next phase and returns to MOVING.
// ============================================================

void startAutomatic() {
  autoRunning  = true;
  autoState    = MOVING;
  activePhase  = 0;
  activeServo  = 0;
  loadPhase(activePhase);
}

void runAutomaticMode() {

  if (!autoRunning) return;

  // ---- PAUSING state ----
  if (autoState == PAUSING) {
    if (millis() - pauseStartTime >= phasePauseMS[activePhase]) {
      // Pause over, advance to next phase
      activePhase = (activePhase + 1) % numPhases();
      loadPhase(activePhase);
      autoState = MOVING;
      Serial.print("Auto: pause done, moving to phase ");
      Serial.println(activePhase);
    }
    return;
  }

  // ---- MOVING state ----
  // Check if current active servo has reached its target
  if (currentPositions[activeServo] != targetPositions[activeServo]) return;

  // This servo is done, advance to next servo
  activeServo++;

  // If all servos in this phase are done
  if (activeServo >= NUM_SERVOS) {

    Serial.print("Auto: phase ");
    Serial.print(activePhase);
    Serial.println(" complete.");

    // ---- Signal control ----
    if (activePhase == 0) {       // After phase 1
      digitalWrite(signalPin, HIGH);
      Serial.println("Signal HIGH (after phase 1)");
    }
    if (activePhase == 2) {       // After phase 3
      digitalWrite(signalPin, LOW);
      Serial.println("Signal LOW (after phase 3)");
    }

    if (phasePauseMS[activePhase] > 0) {
      // Enter pause before loading next phase
      autoState      = PAUSING;
      pauseStartTime = millis();
      Serial.print("Auto: pausing for ");
      Serial.print(phasePauseMS[activePhase]);
      Serial.println("ms.");
    } else {
      // No pause, load next phase immediately
      activePhase = (activePhase + 1) % numPhases();
      loadPhase(activePhase);
    }
  }
}

// ============================================================
// MANUAL MODE
```

```
// ============================================================

void snapshotPotsForManual() {
  for (int i = 0; i < NUM_SERVOS; i++) {
    int raw        = analogRead(potPins[i]);
    potBaseline[i] = map(raw, 0, 1023, 0, 180);
    servoBaseline[i] = currentPositions[i];
  }
  Serial.println("Manual ready. Long press to record a phase.");
}

void runManualMode() {

  for (int i = 0; i < NUM_SERVOS; i++) {

    int potValue = analogRead(potPins[i]);
    targetPositions[i] = map(potValue, 0, 1023, 0, 180);
  }
}

// ============================================================
// SERVO MOTION ENGINE
// ============================================================

void updateServos() {

  if (millis() - lastStepTime < stepDelay) return;
  lastStepTime = millis();

  for (int i = 0; i < NUM_SERVOS; i++) {

    // In auto MOVING state, only step the active servo
    if (!manualMode && autoRunning && autoState == MOVING && !movingToDefault && i != activeServo) continue;

    // During default return, only step the active servo
    if (movingToDefault && i != activeServo) continue;

    if (currentPositions[i] < targetPositions[i]) {
      currentPositions[i]++;
      servos[i].write(currentPositions[i]);
    } else if (currentPositions[i] > targetPositions[i]) {
      currentPositions[i]--;
      servos[i].write(currentPositions[i]);
    }
  }
}

// ============================================================
// BUTTON
// Short press > toggle manual/auto
// Long press  > record position (manual mode only)
// ============================================================

void handleButton() {

  bool buttonState = digitalRead(toggleButton);

  if (lastButtonState == HIGH && buttonState == LOW) {
    pressStartTime = millis();
  }

  if (lastButtonState == LOW && buttonState == HIGH) {

    unsigned long pressDuration = millis() - pressStartTime;

    if (pressDuration >= LONG_PRESS_MS) {
      recordCurrentPosition();
    } else {
      manualMode  = !manualMode;
```

```
    autoRunning = false;

    Serial.print("Mode: ");
    Serial.println(manualMode ? "MANUAL" : "AUTOMATIC");

    digitalWrite(signalPin, LOW);
    moveToDefault();
   }

   delay(300);
  }

  lastButtonState = buttonState;
}

void handlePumpButton() {

  bool buttonState = digitalRead(pumpButton);

  if (lastPumpButtonState == HIGH && buttonState == LOW) {

    pumpState = !pumpState;

    digitalWrite(signalPin, pumpState ? HIGH : LOW);

    Serial.print("Pump: ");
    Serial.println(pumpState ? "INHALE" : "EXHALE");

    delay(200);
  }

  lastPumpButtonState = buttonState;
}
```

pump_code

```
#include <SoftwareSerial.h>

SoftwareSerial armSerial(8,7); // RX, TX

const int suctionPin = 4;
const int releasePin = 3;
const int signalPin = 2;
void setup() {

  armSerial.begin(9600);

  pinMode(signalPin, INPUT);

  pinMode(suctionPin, OUTPUT);
  pinMode(releasePin, OUTPUT);

  initialize();
}

void initialize() {

  digitalWrite(suctionPin, LOW);
  digitalWrite(releasePin, LOW);
}

void inhale() {

  // hold vacuum continuously
```

```
  digitalWrite(suctionPin, HIGH);
  digitalWrite(releasePin, LOW);
}

void exhale() {

  digitalWrite(suctionPin, LOW);

  digitalWrite(releasePin, HIGH);
  delay(1000);          // short air release
  digitalWrite(releasePin, LOW);
}

void loop() {

  if (digitalRead(signalPin) == HIGH) {
    inhale();
  }
  else {
    exhale();
  }

}
```